

1 A view on How to Approach the Project

We need to translate geographical coordinates into mathematical features that an algorithm can understand.

Step 1: Feature Engineering

Algorithms don't inherently understand that $[45.4642, 9.1900]$ coordinates is near a metro stop. we have to create mathematical metrics for it.

- **Distance Metrics:** Calculate the distance from each rental to the nearest metro stop and nearest bike station.
- **Density Metrics:** Count how many bike stations or metro lines are within a specific radius.

Step 2: Exploratory Spatial Data Analysis

Before running models, prove that location matters. Map the rental prices (e.g., using a heatmap).

- Test for **Spatial Autocorrelation**. According to Tobler's First Law of Geography, "everything is related to everything else, but near things are more related than distant things." You should calculate Moran's I to prove statically that expensive rentals cluster near other expensive rentals.

Step 3: Train-Test Split with a Spatial Twist

Standard random splitting can cause "data leakage" in spatial datasets because a house in the training set might be right next door to a house in the test set. Consider **spatial cross-validation** (holding out entire neighborhoods or grids at a time) to truly test your model's predictive power.

2 The Industry Standard (The Baselines)

In professional geostatistical analysis and academic studies, you are expected to compare a few standard models to prove your methodology works.

- **Ordinary Least Squares (OLS) Regression:** This the baseline. we regress price against room properties (size, floor, furnished) and the engineered location features (distance to metro). However, OLS often fails here because it assumes data points are independent which violates spatial autocorrelation.
- **Universal Kriging (Regression-Kriging):** Heavily used in mass appraisal, this combines econometrics with geostatistics. It is a two-step process: first, using a standard regression model (like OLS) to predict prices based on observable features (distance to transit, square footage); second, using Kriging to spatially interpolate the model's errors (residuals). This perfectly captures the unobserved premium across the city.
- **Spatial Autoregressive Models (SAR / SEM):** These are the traditional econometric fixes. They mathematically account for the fact that a room's price is influenced by the prices of its immediate neighbors (Spatial Lag) or unobserved neighborhood factors (Spatial Error).
- **Geographically Weighted Regression (GWR):** GWR understands that the value of a feature changes depending on where you are. For example, being near a metro stop might add a massive premium in the suburbs, but matter less in the highly-walkable city center. GWR calculates different coefficients for different locations.
- **Tree-Based Ensembles:** Random Forests and Gradient Boosting Machines (XGBoost, LightGBM) for predictive accuracy. If we feed them Latitude, Longitude, and distance metrics, they are incredibly good at finding complex, non-linear spatial interactions.

3 Ways to Innovate

We need to use something other than simple distances (euclidian - manhattan) and simple regressions.

- **Isochrones (Network Distance):** Instead of calculating the Euclidean distance to a metro stop, use a routing API (like OSMnx in Python or OpenRouteService) to calculate the actual walking time along the street network. A metro stop 200 meters away isn't helpful if there is an uncrossable highway in between. Walking time is a vastly superior predictor of price.
- **Point of Interest (POI) Embeddings:** Beyond just transport, use OpenStreetMap to download other POIs (cafes, universities, parks, nightlife). You can use unsupervised learning (like PCA or K-Means clustering) to group these into neighborhood profiles, and feed those profiles into the predictive model.
- **Spatial Machine Learning Hybrids:** Combine the interpretability of spatial models with the power of ML. Look into Random Forest Spatial Interpolation (RFSI) or Geographic Random Forests (GRF).